

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITÉ ESPRIT

ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE
ET DE TECHNOLOGIES



Rapport de Projet d'Intelligence Artificielle Embarquée

Détection d'émotions faciales

Réalisé par :

- Ghachen Aziz
- Souki Aya
- Brahem Naceur

Année universitaire : 2025 – 2026

Table des matières

Liste des abréviations	iv
INTRODUCTION GÉNÉRALE	1
1 PRÉSENTATION GÉNÉRALE DU PROJET	2
1.1 Introduction	2
1.2 L'intelligence artificielle	2
1.2.1 Définition	2
1.2.2 Avantages et limites	3
1.3 Machine learning et deep learning	4
1.3.1 Machine learning	4
1.3.2 Deep learning	4
1.3.3 Réseaux de neurones convolutifs	4
1.4 Reconnaissance d'émotions faciales	5
1.5 Contraintes des systèmes embarqués pour l'intelligence artificielle	5
1.6 Problématique et objectifs du projet	6
1.7 Conclusion	6
2 CONCEPTION DU SYSTÈME	7
2.1 Introduction	7
2.2 Présentation générale de l'architecture du système	7
2.3 Choix des logiciels et outils	8
2.3.1 Environnement de développement Python	8
2.3.2 TensorFlow / Keras	8
2.3.3 Environnement d'entraînement (PC / Google Colab)	9
2.4 Phase d'apprentissage	9
2.4.1 Jeu de données utilisé	9
2.4.2 Préparation des données	9
2.5 Architecture du réseau de neurones convolutif	10
2.5.1 Diagramme CNN	10
2.6 Entraînement du modèle	11
2.7 Réglage des hyperparamètres	11

2.8	Évaluation expérimentale sur Google Colab	11
2.9	Analyse des résultats	15
2.10	Conclusion	15
3	Déploiement et résultats sur système embarqué	16
3.1	Introduction	16
3.2	Présentation de la plateforme matérielle	16
3.3	Préparation des données d'entrée pour le système embarqué	17
3.3.1	Conversion des images en tableaux C	17
3.4	Déploiement du modèle avec STM32CubeMX	17
3.4.1	Création du projet à partir de la carte	17
3.4.2	Configuration du microcontrôleur	18
3.4.3	Installation et activation du middleware X-CUBE-AI	19
3.4.4	Importation et analyse du modèle	19
3.4.5	Analyse et validation du modèle	20
3.4.6	Génération du code	21
3.5	Exploitation du projet dans STM32CubeIDE	21
3.5.1	Importation, compilation et flashage	21
3.5.2	Configuration de la communication série (UART1)	22
3.5.3	Exécution de l'inférence avec image statique	22
3.6	Amélioration : affichage du résultat sur l'écran LCD	23
3.6.1	Initialisation de l'écran LCD	23
3.6.2	Affichage de l'émotion prédite	23
3.7	Conclusion	23
	CONCLUSION GÉNÉRALE	24
	Bibliographie	25

Table des figures

1.1	Hiérarchie de l'intelligence artificielle	3
1.2	Architecture générale d'un réseau CNN appliqué à la classification d'images	5
2.1	Architecture globale du système de détection d'émotions	8
2.2	Exemples d'images du dataset (48×48, niveaux de gris)	9
2.3	Courbes d'apprentissage : Accuracy et Loss	12
2.4	Rapport de classification du modèle	12
2.5	Matrice de confusion sur l'ensemble de test	13
2.6	Exemple de prédiction du modèle pour une image exprimant la surprise, avec un taux de confiance élevé	14
2.7	Exemple de prédiction du modèle pour une image exprimant la tristesse, avec un taux de confiance modéré	14
3.1	Sélection de la carte STM32F746G-DISCO dans STM32CubeMX	18
3.2	Vue Pinout & Configuration du microcontrôleur STM32F746	18
3.3	Sélection et installation du pack logiciel X-CUBE-AI	19
3.4	Importation et analyse du modèle Keras dans STM32CubeMX	20
3.5	Analyse de la complexité et validation du modèle dans STM32CubeMX	20
3.6	Projet généré par STM32CubeMX compilé avec succès dans STM32CubeIDE	21
3.7	Affichage des résultats d'inférence via l'interface UART sur PuTTY	22

Liste des tableaux

2.1	Architecture du modèle CNN pour la détection des émotions faciales	10
-----	--	----

Liste des abréviations

IA	Intelligence Artificielle
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
MCU	Microcontrôleur
UART	Universal Asynchronous Receiver/Transmitter
BSP	Board Support Package

INTRODUCTION GÉNÉRALE

L'intelligence artificielle est une technologie qui permet aux machines d'exécuter des tâches nécessitant habituellement une intervention humaine, en s'appuyant sur des algorithmes capables d'apprendre à partir des données et de s'adapter à différents contextes. Ces dernières années, les avancées en apprentissage automatique et en apprentissage profond ont permis le développement de systèmes performants dans des domaines variés tels que la vision par ordinateur, le traitement du langage ou encore la reconnaissance de formes.

L'application des techniques d'apprentissage profond à la reconnaissance d'émotions faciales constitue aujourd'hui un enjeu important, notamment dans les domaines de l'interaction homme-machine, de la sécurité ou des systèmes intelligents d'assistance. Ce projet a pour objectif de concevoir un système capable de détecter automatiquement les émotions faciales à partir d'images, tout en tenant compte des contraintes spécifiques liées à une exécution sur un système embarqué.

La première étape du projet consiste à collecter et préparer un ensemble de données d'images faciales afin d'entraîner un modèle de deep learning capable de reconnaître différentes classes d'émotions. Ces données sont ensuite utilisées pour concevoir et entraîner un réseau de neurones convolutif adapté à la reconnaissance d'images, tout en veillant à limiter la complexité du modèle pour permettre son déploiement sur une plateforme embarquée.

Ce document est structuré en plusieurs parties. La première partie présente les concepts généraux liés à l'intelligence artificielle, au machine learning et au deep learning, ainsi que les principes fondamentaux de la reconnaissance d'images. La deuxième partie est consacrée à la conception du système, en détaillant l'architecture du modèle utilisé, les outils logiciels mobilisés et les différentes étapes de l'entraînement et de l'évaluation du réseau de neurones. La troisième partie décrit le déploiement du modèle sur un microcontrôleur STM32, en mettant en évidence les choix matériels et logiciels effectués ainsi que les résultats obtenus lors de l'exécution en conditions réelles. Enfin, une conclusion générale permet de synthétiser les différentes étapes du projet, d'analyser les résultats obtenus et de souligner les apports de ce travail.

Chapitre 1

PRÉSENTATION GÉNÉRALE DU PROJET

1.1 Introduction

L'intelligence artificielle connaît depuis plusieurs années un développement rapide, porté par l'augmentation des capacités de calcul, la disponibilité croissante des données et les progrès réalisés dans les algorithmes d'apprentissage automatique. Ces avancées ont permis l'émergence de solutions performantes dans des domaines variés tels que la vision par ordinateur, le traitement du langage naturel ou encore les systèmes intelligents embarqués.

Parmi les applications de la vision par ordinateur, la reconnaissance d'émotions faciales occupe une place importante. Elle vise à analyser automatiquement les expressions du visage afin d'identifier l'état émotionnel d'un individu. Ce type de technologie est aujourd'hui utilisé dans des domaines tels que l'interaction homme-machine, l'assistance intelligente ou encore l'analyse comportementale.

Ce projet s'inscrit dans ce contexte et vise à concevoir un système de détection d'émotions faciales basé sur des techniques d'intelligence artificielle, tout en tenant compte des contraintes spécifiques liées à une exécution sur un système embarqué. Ce chapitre a pour objectif de présenter les concepts fondamentaux nécessaires à la compréhension du projet, ainsi que les enjeux et la problématique associée.

1.2 L'intelligence artificielle

1.2.1 Définition

L'intelligence artificielle (IA) désigne l'ensemble des techniques permettant à une machine de simuler certaines capacités cognitives humaines, telles que la perception, le raisonnement, l'apprentissage ou la prise de décision. Elle repose sur des algorithmes capables

d'analyser des données, d'en extraire des informations pertinentes et d'adapter leur comportement en fonction des résultats obtenus.

Les domaines d'application de l'intelligence artificielle sont nombreux et en constante évolution. On peut notamment citer la reconnaissance vocale et faciale, l'aide à la décision, la robotique, les systèmes de recommandation ou encore la conduite autonome. Ces applications démontrent le potentiel de l'IA à résoudre des problèmes complexes nécessitant traditionnellement une intervention humaine.

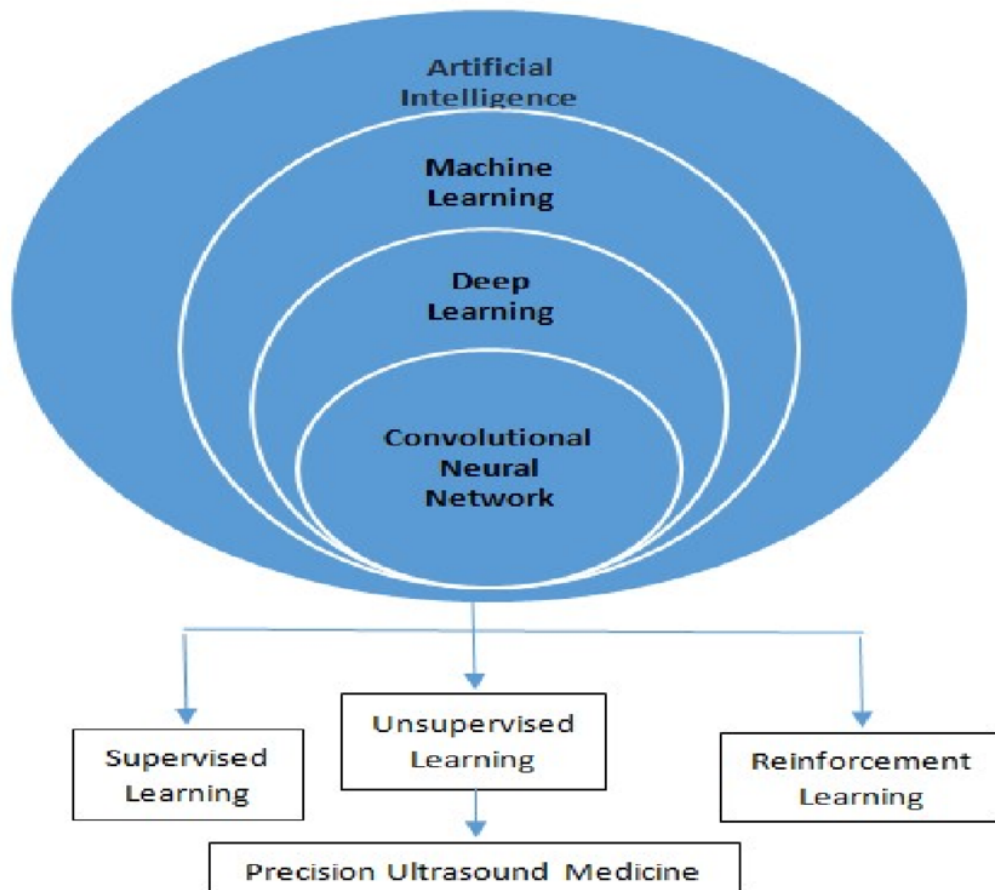


FIGURE 1.1 – Hiérarchie de l'intelligence artificielle

1.2.2 Avantages et limites

L'un des principaux avantages de l'intelligence artificielle réside dans sa capacité à traiter de grandes quantités de données et à automatiser des tâches complexes avec un haut niveau de précision. Elle permet également d'améliorer la performance et l'efficacité de nombreux systèmes, tout en réduisant les erreurs humaines.

Cependant, l'intelligence artificielle présente également certaines limites. La qualité des résultats dépend fortement de la qualité et de la représentativité des données utilisées lors de l'apprentissage. De plus, les modèles les plus performants nécessitent souvent des ressources importantes en termes de calcul et de mémoire, ce qui peut poser problème

dans un contexte embarqué.

1.3 Machine learning et deep learning

1.3.1 Machine learning

Le machine learning, ou apprentissage automatique, est une branche de l'intelligence artificielle qui consiste à concevoir des algorithmes capables d'apprendre automatiquement à partir des données. Contrairement aux approches classiques basées sur des règles explicites, les modèles de machine learning identifient des motifs et des relations dans les données afin de réaliser des prédictions ou des classifications.

On distingue principalement trois types d'apprentissage : l'apprentissage supervisé, l'apprentissage non supervisé et l'apprentissage par renforcement. Dans le cadre de ce projet, l'apprentissage supervisé est utilisé, car les images faciales sont associées à des étiquettes correspondant aux émotions à reconnaître.

1.3.2 Deep learning

Le deep learning est une sous-catégorie du machine learning reposant sur l'utilisation de réseaux de neurones artificiels profonds, composés de plusieurs couches successives. Ces architectures permettent de modéliser des relations complexes et sont particulièrement adaptées au traitement de données non structurées, telles que les images.

Grâce à leur capacité à extraire automatiquement des caractéristiques pertinentes à partir des données brutes, les modèles de deep learning sont largement utilisés en vision par ordinateur. Ils permettent notamment d'obtenir de bonnes performances pour des tâches telles que la classification d'images ou la reconnaissance faciale.

1.3.3 Réseaux de neurones convolutifs

Les réseaux de neurones convolutifs (CNN) constituent une architecture de deep learning particulièrement adaptée à l'analyse d'images. Ils reposent sur des couches convolutives capables d'extraire des caractéristiques locales, telles que les contours ou les motifs, suivies de couches de réduction de dimension et de classification.

Dans le cadre de la reconnaissance d'émotions faciales, les CNN permettent de capturer les variations subtiles des expressions du visage, ce qui en fait un choix pertinent pour ce type d'application.

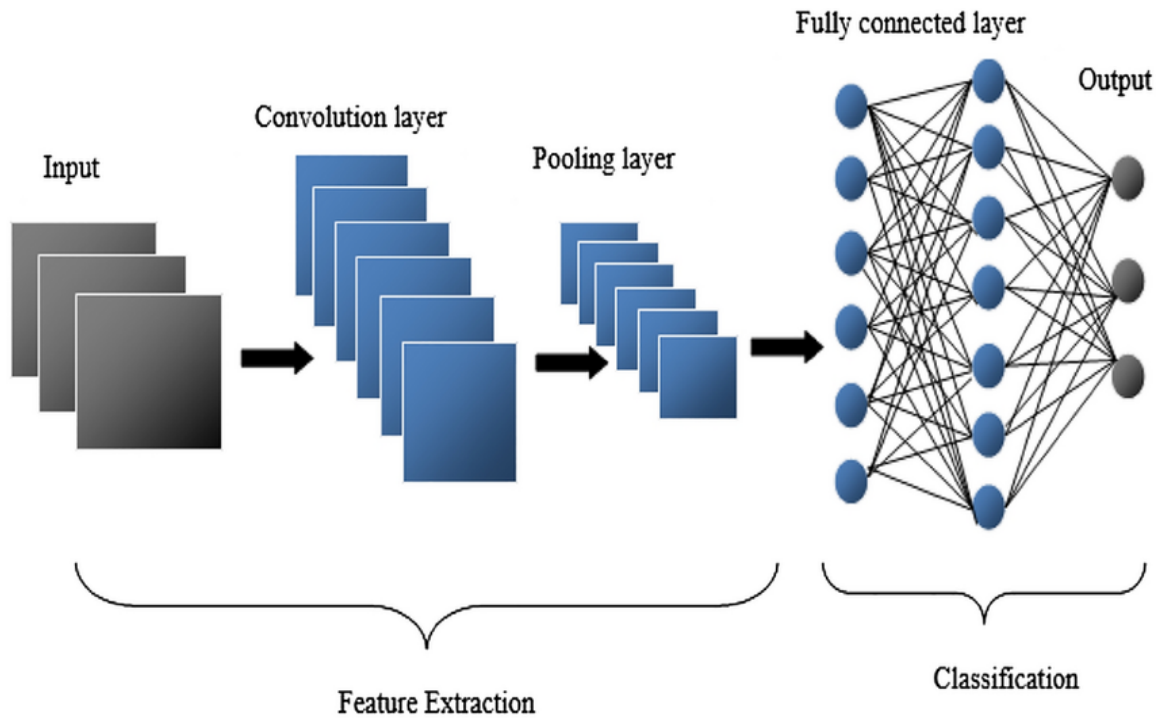


FIGURE 1.2 – Architecture générale d’un réseau CNN appliqué à la classification d’images

1.4 Reconnaissance d’émotions faciales

La reconnaissance d’émotions faciales vise à identifier automatiquement l’émotion exprimée par un individu à partir de l’analyse de son visage. Les émotions les plus couramment étudiées incluent la joie, la tristesse, la colère, la surprise, la peur, le dégoût et l’état neutre.

Cette tâche est complexe, car les expressions faciales peuvent varier considérablement d’un individu à l’autre et être influencées par de nombreux facteurs, tels que l’éclairage, l’angle de vue ou la qualité de l’image. Malgré ces difficultés, les approches basées sur le deep learning ont permis d’améliorer significativement les performances des systèmes de reconnaissance d’émotions.

1.5 Contraintes des systèmes embarqués pour l’intelligence artificielle

Contrairement aux environnements de calcul classiques tels que les ordinateurs ou les serveurs, les systèmes embarqués imposent des contraintes strictes en termes de ressources. Ces contraintes concernent principalement la mémoire disponible, la puissance de calcul et la consommation énergétique.

Dans le cas d’un microcontrôleur STM32, la mémoire RAM et la mémoire Flash sont limitées, ce qui restreint la taille et la complexité des modèles d’intelligence artificielle

pouvant être déployés. De plus, la puissance de calcul est inférieure à celle d'un processeur ou d'un GPU, ce qui impose une optimisation des opérations réalisées lors de l'inférence.

Ces contraintes influencent directement le choix de l'architecture du modèle, la résolution des données en entrée et les techniques d'optimisation utilisées. Il est donc nécessaire de trouver un compromis entre les performances du modèle et sa compatibilité avec la plateforme embarquée.

1.6 Problématique et objectifs du projet

La plupart des systèmes de reconnaissance d'émotions faciales sont développés et déployés sur des plateformes disposant de ressources importantes. Leur intégration dans des systèmes embarqués à ressources limitées reste donc un défi technique majeur.

La problématique centrale de ce projet peut ainsi être formulée comme suit :

comment concevoir et déployer un modèle de deep learning pour la reconnaissance d'émotions faciales sur un microcontrôleur STM32, tout en respectant des contraintes strictes de mémoire, de calcul et de performances ?

L'objectif du projet est de développer un modèle de reconnaissance d'émotions faciales capable de fonctionner sur une plateforme embarquée, en adaptant l'architecture du réseau et les données d'entrée aux contraintes du microcontrôleur.

1.7 Conclusion

Ce chapitre a permis de présenter les concepts fondamentaux liés à l'intelligence artificielle, au machine learning et au deep learning, ainsi que les principes généraux de la reconnaissance d'émotions faciales. Il a également mis en évidence les contraintes spécifiques associées aux systèmes embarqués et la problématique centrale du projet.

Ces éléments constituent le cadre théorique et méthodologique sur lequel repose la suite du travail. Le chapitre suivant sera consacré à la conception du système et à la description détaillée des choix techniques effectués pour l'entraînement et l'évaluation du modèle de deep learning.

Chapitre 2

CONCEPTION DU SYSTÈME

2.1 Introduction

Ce chapitre est consacré à la conception du système de détection d'émotions faciales et à l'entraînement du modèle de deep learning. Il vise à présenter la méthodologie adoptée, les choix techniques effectués ainsi que les étapes de préparation des données, d'entraînement et d'évaluation du modèle.

Dans un contexte d'intelligence artificielle embarquée, la conception du modèle doit répondre à un double objectif : garantir des performances satisfaisantes en reconnaissance d'émotions tout en respectant les contraintes de ressources imposées par le microcontrôleur STM32. Les choix présentés dans ce chapitre s'inscrivent dans cette logique de compromis entre précision et complexité.

2.2 Présentation générale de l'architecture du système

L'architecture globale du système repose sur une chaîne de traitement structurée en plusieurs étapes. Dans un premier temps, les images faciales sont prétraitées afin d'être compatibles avec l'entrée du réseau de neurones. Ces images sont ensuite transmises à un modèle de type réseau de neurones convolutif, chargé d'extraire automatiquement les caractéristiques pertinentes et de réaliser la classification de l'émotion.

Le processus d'apprentissage du modèle est réalisé hors ligne sur un ordinateur disposant de ressources de calcul suffisantes. Une fois entraîné et validé, le modèle est ensuite exporté et intégré dans l'environnement embarqué, où seule la phase d'inférence est exécutée. Cette séparation entre apprentissage et inférence permet de limiter la charge de calcul sur le microcontrôleur.

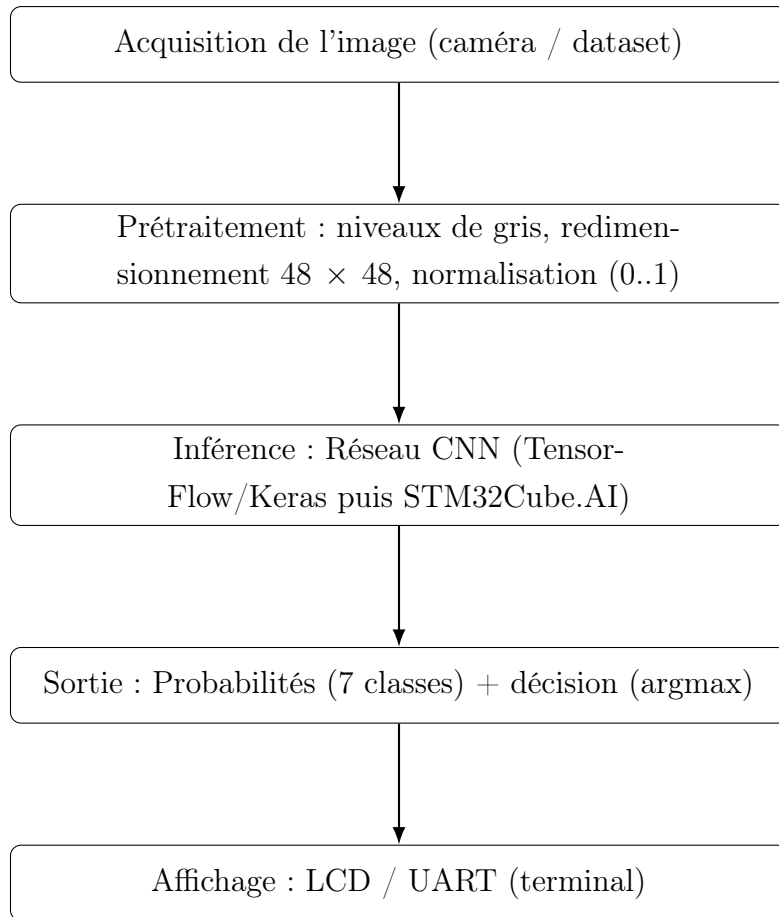


FIGURE 2.1 – Architecture globale du système de détection d’émotions

2.3 Choix des logiciels et outils

2.3.1 Environnement de développement Python

Le langage Python a été utilisé pour l’ensemble des étapes de conception et d’entraînement du modèle. Il offre une grande flexibilité et dispose de nombreuses bibliothèques dédiées à l’intelligence artificielle et au traitement des données. L’utilisation de Python permet également un prototypage rapide et une expérimentation efficace des différentes architectures de modèles.

2.3.2 TensorFlow / Keras

Les bibliothèques TensorFlow et Keras ont été utilisées pour la conception et l’entraînement du réseau de neurones convolutif. Keras fournit une interface de haut niveau facilitant la définition des architectures de réseaux, tandis que TensorFlow assure l’exécution optimisée des calculs nécessaires à l’apprentissage.

Le choix de ces outils est également motivé par leur compatibilité avec les solutions de déploiement embarqué proposées par STMicroelectronics, notamment via STM32Cube.AI

2.3.3 Environnement d'entraînement (PC / Google Colab)

L'entraînement du modèle a été réalisé sur Google Colab, qui offre un environnement de calcul accessible et performant. Cette plateforme permet de disposer de ressources suffisantes pour entraîner des modèles de deep learning sans contrainte matérielle locale.

2.4 Phase d'apprentissage

2.4.1 Jeu de données utilisé

Le modèle est entraîné sur un dataset d'expressions faciales en niveaux de gris (48×48). Dans ce projet :

- **Entraînement** : 28 709 images (7 classes)
- **Test** : 7 178 images (7 classes)

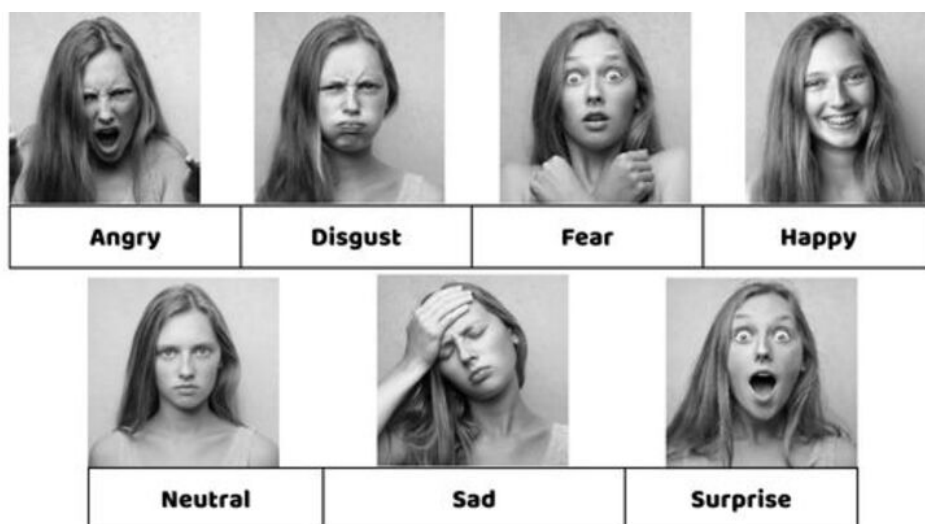


FIGURE 2.2 – Exemples d'images du dataset (48×48 , niveaux de gris)

2.4.2 Préparation des données

Avant l'entraînement, les données ont été soumises à plusieurs étapes de prétraitement. Les images ont été nettoyées afin d'éliminer les données corrompues ou de mauvaise qualité. Les valeurs de pixels ont ensuite été normalisées afin d'améliorer la stabilité de l'apprentissage.

Le jeu de données a été divisé en trois ensembles distincts : un ensemble d'entraînement, un ensemble de validation et un ensemble de test. Cette séparation permet d'évaluer objectivement les performances du modèle et de limiter les risques de surapprentissage.

2.5 Architecture du réseau de neurones convolutif

Le modèle retenu repose sur une architecture de réseau de neurones convolutif légère. Ce choix est motivé par la nécessité de limiter la complexité du modèle afin de garantir sa compatibilité avec les contraintes du microcontrôleur STM32.

Les couches convolutives permettent d'extraire automatiquement des caractéristiques locales à partir des images faciales, tandis que les couches de pooling réduisent la dimension des données et le nombre de paramètres. Enfin, des couches entièrement connectées assurent la classification finale de l'émotion.

Le compromis entre profondeur du réseau et performances est un élément central de la conception du modèle. Une architecture trop complexe améliorerait potentiellement la précision, mais rendrait le déploiement embarqué difficile, voire impossible.

2.5.1 Diagramme CNN

Le modèle proposé est un réseau de neurones convolutif composé de quatre blocs de convolution suivis d'une normalisation par lots et d'un sous-échantillonnage. Une couche de Global Average Pooling est utilisée afin de réduire le nombre de paramètres et d'éviter le sur-apprentissage. Enfin, une couche entièrement connectée avec une fonction d'activation Softmax permet la classification des émotions en sept classes.

Bloc	Type de couche	Paramètres	Sortie
1	Conv2D + ReLU	16 filtres, 3×3	$48 \times 48 \times 16$
	Batch Normalization	–	$48 \times 48 \times 16$
	MaxPooling2D	2×2	$24 \times 24 \times 16$
2	Conv2D + ReLU	32 filtres, 3×3	$24 \times 24 \times 32$
	Batch Normalization	–	$24 \times 24 \times 32$
	MaxPooling2D	2×2	$12 \times 12 \times 32$
3	Conv2D + ReLU	64 filtres, 3×3	$12 \times 12 \times 64$
	Batch Normalization	–	$12 \times 12 \times 64$
	MaxPooling2D	2×2	$6 \times 6 \times 64$
4	Conv2D + ReLU	128 filtres, 3×3	$6 \times 6 \times 128$
	Batch Normalization	–	$6 \times 6 \times 128$
5	Global Average Pooling	–	128
6	Dense (Softmax)	7 neurones	7 classes

TABLE 2.1 – Architecture du modèle CNN pour la détection des émotions faciales

2.6 Entraînement du modèle

L'entraînement du modèle a été réalisé de manière non supervisée (images sans labels). Une fonction de perte de type entropie croisée a été utilisée afin de mesurer l'écart entre les prédictions du modèle et les étiquettes réelles. Un algorithme d'optimisation (Adam) a permis d'ajuster progressivement les poids du réseau de neurones. L'apprentissage s'est déroulé sur plusieurs époques, permettant au modèle de converger vers une solution satisfaisante.

2.7 Réglage des hyperparamètres

Les hyperparamètres ajustés sont :

- learning rate = 0.001 ,
- batch size = 64,
- nombre d'époques = 60,
- profondeur du réseau = 13 couches,

ainsi que les callbacks (EarlyStopping et ReduceLROnPlateau).

2.8 Évaluation expérimentale sur Google Colab

Afin de valider les performances du modèle avant son déploiement sur la plateforme embarquée, une phase de test a été réalisée dans l'environnement Google Colab. Cette étape permet d'évaluer le comportement du modèle dans des conditions contrôlées, à partir d'images issues du jeu de test, et d'analyser plus finement les performances par classe d'émotion.

Les courbes d'apprentissage

Les courbes d'apprentissage (Figure 2.3) montrent une convergence progressive du modèle, avec une stabilisation de la perte et de la précision sur l'ensemble de validation, indiquant une absence de surapprentissage significatif.

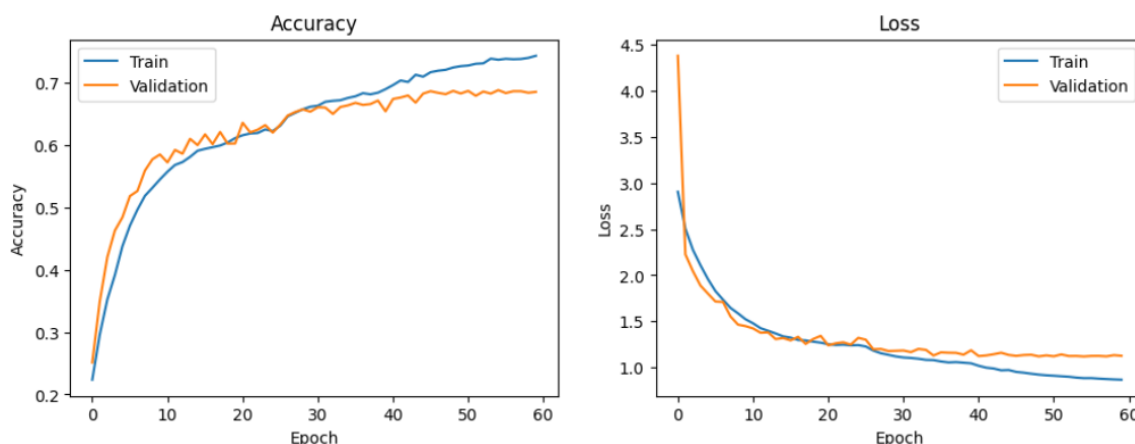


FIGURE 2.3 – Courbes d’apprentissage : Accuracy et Loss

Rapport de classification

L’évaluation quantitative du modèle a été réalisée à l’aide d’un rapport de classification. Ce rapport présente, pour chaque classe d’émotion, les métriques de précision (*precision*), de rappel (*recall*) et de score F1 (*f1-score*), ainsi que le nombre d’échantillons associés (*support*).

Classification Report:

	precision	recall	f1-score	support
angry	0.61	0.60	0.60	958
disgusted	0.68	0.59	0.63	111
fearful	0.58	0.43	0.49	1024
happy	0.86	0.88	0.87	1774
neutral	0.58	0.70	0.64	1233
sad	0.56	0.58	0.57	1247
surprised	0.80	0.77	0.79	831
accuracy			0.68	7178
macro avg	0.67	0.65	0.66	7178
weighted avg	0.68	0.68	0.68	7178

FIGURE 2.4 – Rapport de classification du modèle

L’analyse de ce rapport montre que le modèle obtient de très bonnes performances pour certaines émotions bien marquées, notamment la joie (*happy*) et la surprise (*surprised*), avec des scores F1 respectivement élevés. Ces résultats s’expliquent par la forte expressivité

faciale associée à ces émotions, qui facilite leur identification par le réseau de neurones convolutif.

En revanche, certaines émotions telles que la peur (*fearful*), la tristesse (*sad*) ou l'état neutre (*neutral*) présentent des scores plus faibles. Ces confusions sont principalement dues à la similarité des expressions faciales et à la faible résolution des images d'entrée (48×48 pixels), qui limite la capture de détails subtils du visage.

La précision globale du modèle atteint environ **68 %**, ce qui constitue un compromis satisfaisant compte tenu des contraintes imposées par un futur déploiement sur microcontrôleur.

Matrice de confusion

La Figure 2.5 illustre la matrice de confusion obtenue sur l'ensemble de test. Elle permet d'identifier les classes d'émotions les plus confondues et d'évaluer la robustesse du modèle pour chaque catégorie.

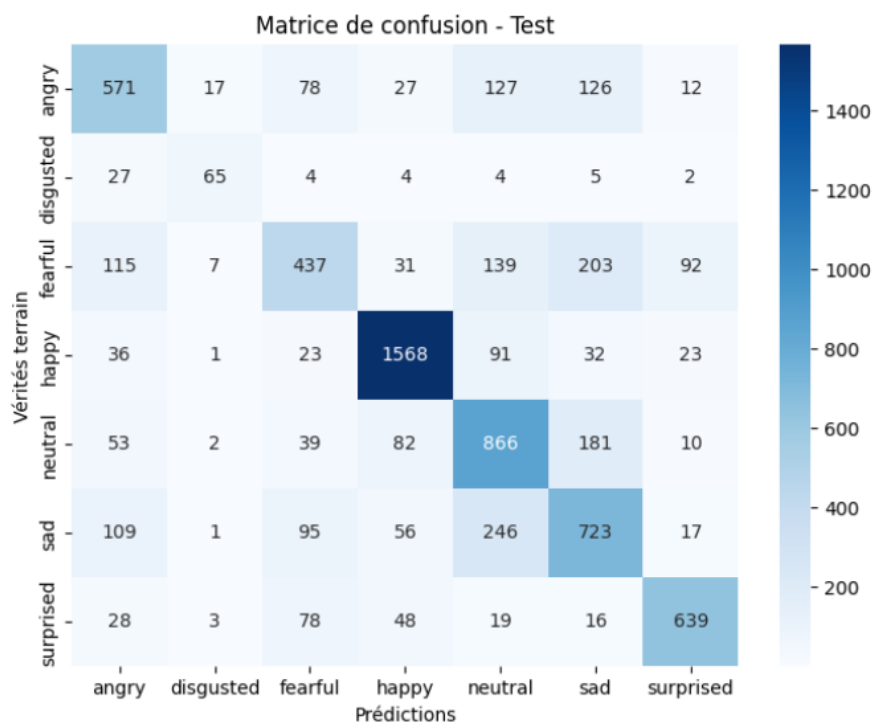


FIGURE 2.5 – Matrice de confusion sur l'ensemble de test

Tests du modèle sur des images individuelles

En complément de l'évaluation quantitative, des tests qualitatifs ont été réalisés sur des images individuelles afin d'observer le comportement du modèle en situation réelle. Pour chaque image testée, le modèle fournit une émotion prédite accompagnée d'un taux de confiance.



FIGURE 2.6 – Exemple de prédiction du modèle pour une image exprimant la surprise, avec un taux de confiance élevé

Dans le premier exemple, le modèle identifie correctement l'émotion de surprise avec un taux de confiance supérieur à 98 %. Ce résultat met en évidence la capacité du réseau à reconnaître efficacement des expressions faciales fortement marquées.

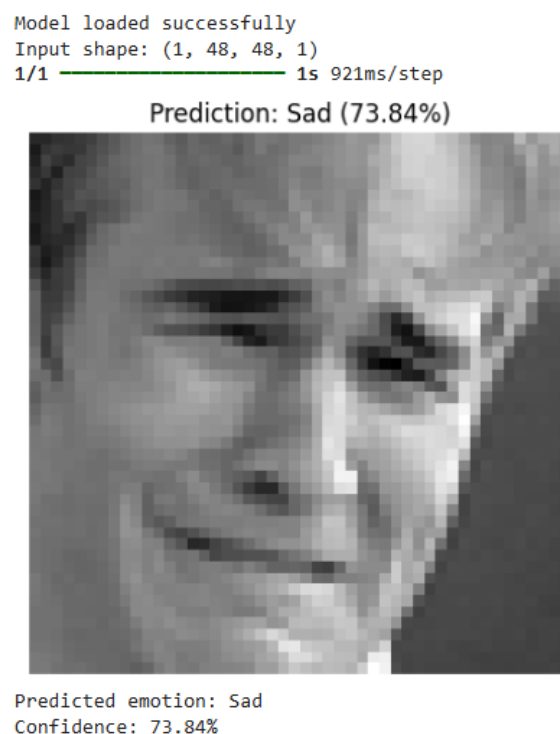


FIGURE 2.7 – Exemple de prédiction du modèle pour une image exprimant la tristesse, avec un taux de confiance modéré

Le second exemple illustre une prédiction correcte de l'émotion de tristesse, avec un taux de confiance d'environ 74 %. Bien que la prédiction soit correcte, le niveau de confiance plus faible reflète la difficulté du modèle à distinguer certaines émotions proches, notamment lorsque les expressions faciales sont moins accentuées.

2.9 Analyse des résultats

Les résultats obtenus sur Google Colab confirment que le modèle est globalement capable de reconnaître les principales émotions faciales à partir d'images en niveaux de gris de faible résolution. Les performances sont particulièrement satisfaisantes pour les émotions présentant des caractéristiques visuelles distinctes.

Cependant, certaines limites subsistent. Les confusions observées entre des émotions proches mettent en évidence l'impact de la résolution réduite des images et du déséquilibre du jeu de données. Ces limites sont acceptables dans le cadre de ce projet, dont l'objectif principal est de démontrer la faisabilité du déploiement d'un modèle de deep learning sur une plateforme embarquée à ressources limitées.

Cette phase de validation sur Google Colab constitue une étape essentielle avant l'intégration du modèle dans l'environnement STM32, présentée dans le chapitre suivant.

2.10 Conclusion

Ce chapitre a présenté la conception complète du modèle de détection d'émotions faciales, depuis le choix des outils et des données jusqu'à l'analyse des performances obtenues. Les choix méthodologiques effectués s'inscrivent dans une démarche visant à concilier performances et contraintes matérielles.

Chapitre 3

Déploiement et résultats sur système embarqué

3.1 Introduction

Après la phase de conception et d'entraînement du modèle de deep learning, l'étape finale du projet consiste à déployer ce modèle sur une plateforme embarquée réelle. Cette phase a pour objectif de valider la faisabilité de l'exécution d'un modèle de reconnaissance d'émotions faciales sur un microcontrôleur STM32, en tenant compte des contraintes de mémoire et de capacité de calcul propres aux systèmes embarqués.

Contrairement à une approche basée sur l'acquisition d'images en temps réel à l'aide d'une caméra, les tests réalisés dans ce projet reposent sur l'utilisation d'images statiques. Ces images ont été préalablement converties en tableaux de données en langage C, puis intégrées directement dans le code embarqué. Cette méthode permet de tester et de valider le fonctionnement du modèle de manière contrôlée, tout en conservant un format d'entrée identique à celui utilisé lors de l'entraînement.

Le déploiement du modèle s'effectue en deux étapes principales :

- une première étape de configuration et d'intégration du modèle à l'aide de STM32CubeMX ;
- une seconde étape de compilation, d'exécution et de test du programme dans STM32CubeIDE.

3.2 Présentation de la plateforme matérielle

La plateforme matérielle utilisée est la carte **STM32F746G-DISCO**, basée sur le microcontrôleur **STM32F746NGH6** intégrant un cœur **ARM Cortex-M7**. Ce microcontrôleur offre un bon compromis entre performance de calcul et consommation énergétique, ce qui le rend adapté aux applications d'intelligence artificielle embarquée de complexité modérée.

La carte STM32F746G-DISCO intègre notamment :

- une interface de débogage *ST-Link* permettant la programmation et le débogage ;
- un écran LCD embarqué facilitant l’affichage des résultats ;
- des interfaces de communication série (UART) assurant l’échange de données avec un terminal externe.

Ces caractéristiques font de cette carte une plateforme de test pertinente pour valider le déploiement d’un modèle de deep learning sur microcontrôleur.

3.3 Préparation des données d’entrée pour le système embarqué

3.3.1 Conversion des images en tableaux C

Les images utilisées pour les tests sont des images faciales en niveaux de gris de taille 48×48 pixels, correspondant au format d’entrée du modèle de deep learning. Afin de pouvoir exploiter ces images sur le microcontrôleur STM32, elles ont été converties en tableaux numériques en langage C.

Cette conversion a été réalisée à l’aide d’un script Python exécuté sur Google Colab. Le script permet de :

- charger une image faciale en niveaux de gris ;
- normaliser les valeurs des pixels ;
- transformer l’image en un tableau unidimensionnel de taille 48×48 ;
- générer automatiquement un tableau C intégrable dans le code embarqué.

Cette approche permet de s’affranchir de la nécessité d’une acquisition d’images en temps réel, tout en conservant un format d’entrée strictement identique à celui utilisé lors de l’apprentissage du modèle.

3.4 Déploiement du modèle avec STM32CubeMX

3.4.1 Création du projet à partir de la carte

La première étape du déploiement consiste à créer un nouveau projet dans STM32CubeMX en sélectionnant la carte **STM32F746G-DISCO** via l’option *Board Selector*. Cette sélection permet de charger automatiquement les paramètres matériels associés à la carte.

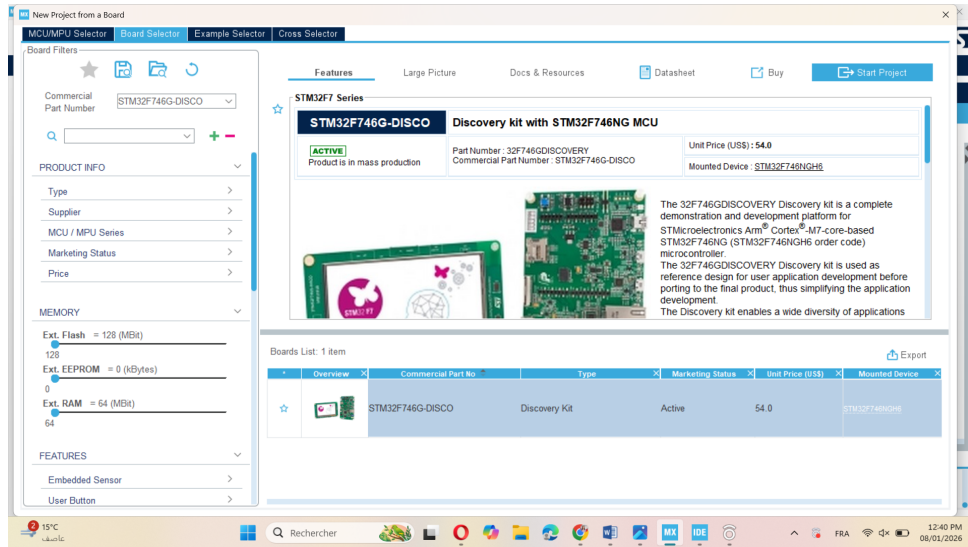


FIGURE 3.1 – Sélection de la carte STM32F746G-DISCO dans STM32CubeMX

3.4.2 Configuration du microcontrôleur

Après la création du projet, la vue *Pinout & Configuration* permet de visualiser et de configurer les périphériques nécessaires. Dans ce projet, la configuration est volontairement minimale et inclut principalement :

- l'activation de l'interface UART pour l'affichage des résultats ;
- la configuration de l'horloge système ;
- l'activation éventuelle de FreeRTOS pour la gestion des tâches.

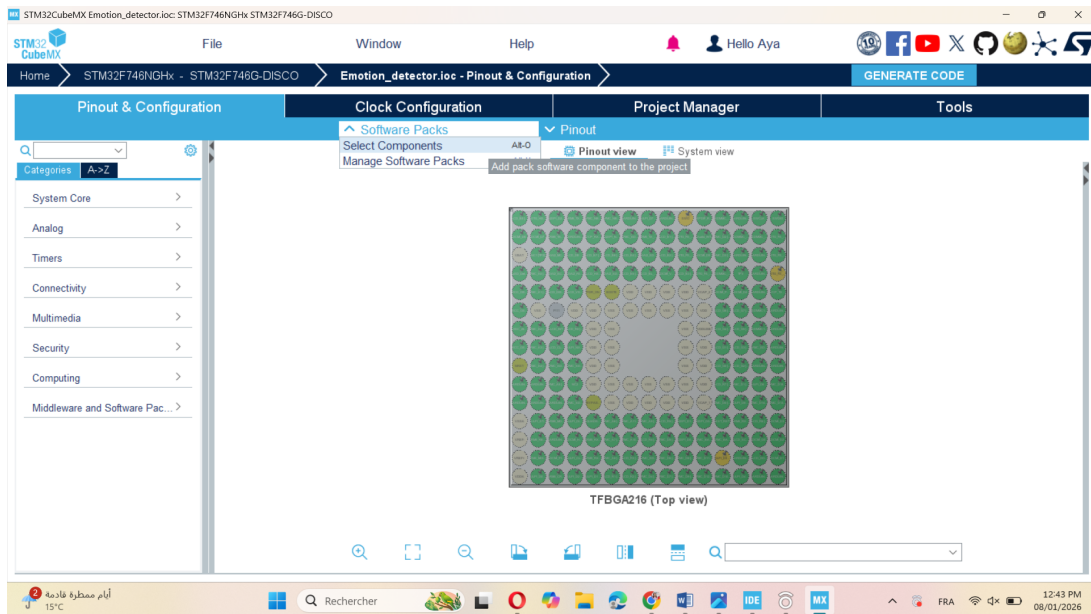


FIGURE 3.2 – Vue Pinout & Configuration du microcontrôleur STM32F746

3.4.3 Installation et activation du middleware X-CUBE-AI

Afin d'intégrer des fonctionnalités d'intelligence artificielle, le pack logiciel **X-CUBE-AI** doit être installé via le gestionnaire de composants logiciels (*Software Packs*). Ce pack permet de convertir et d'exécuter des modèles de deep learning sur microcontrôleur STM32.

Une fois activé, le module *Artificial Intelligence – X-CUBE-AI* devient disponible dans STM32CubeMX et permet la gestion des réseaux de neurones.

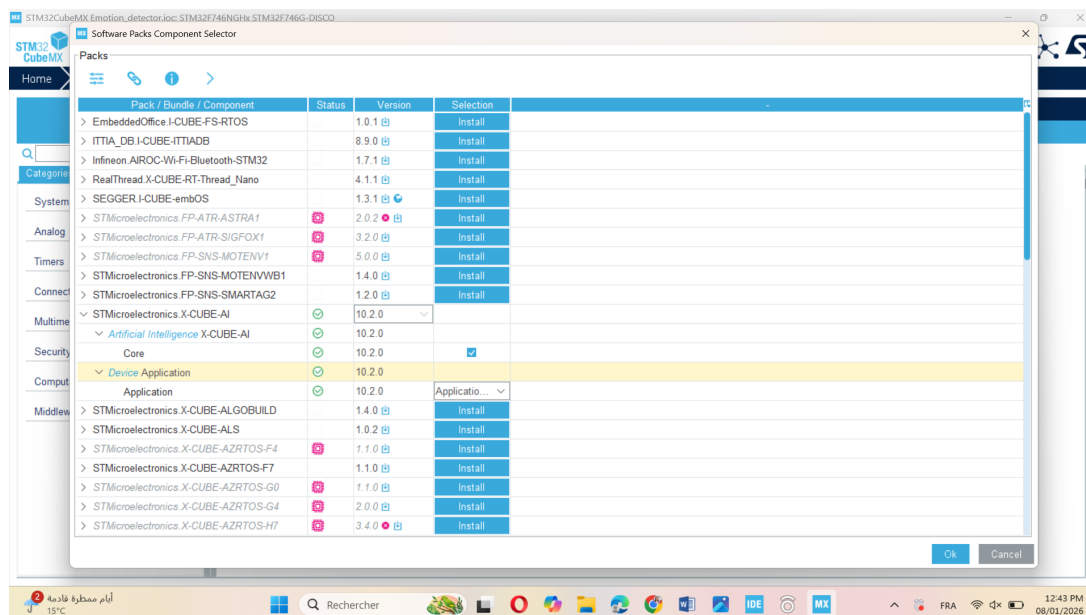


FIGURE 3.3 – Sélection et installation du pack logiciel X-CUBE-AI

3.4.4 Importation et analyse du modèle

Le modèle de reconnaissance d'émotions, entraîné à l'aide de TensorFlow/Keras, est importé dans STM32CubeMX via le module X-CUBE-AI. L'outil analyse automatiquement le modèle et fournit des informations détaillées sur :

- l'utilisation de la mémoire Flash ;
- l'utilisation de la mémoire RAM ;
- la complexité du réseau en nombre d'opérations.

Cette analyse permet de vérifier la compatibilité du modèle avec les ressources disponibles sur la carte STM32.

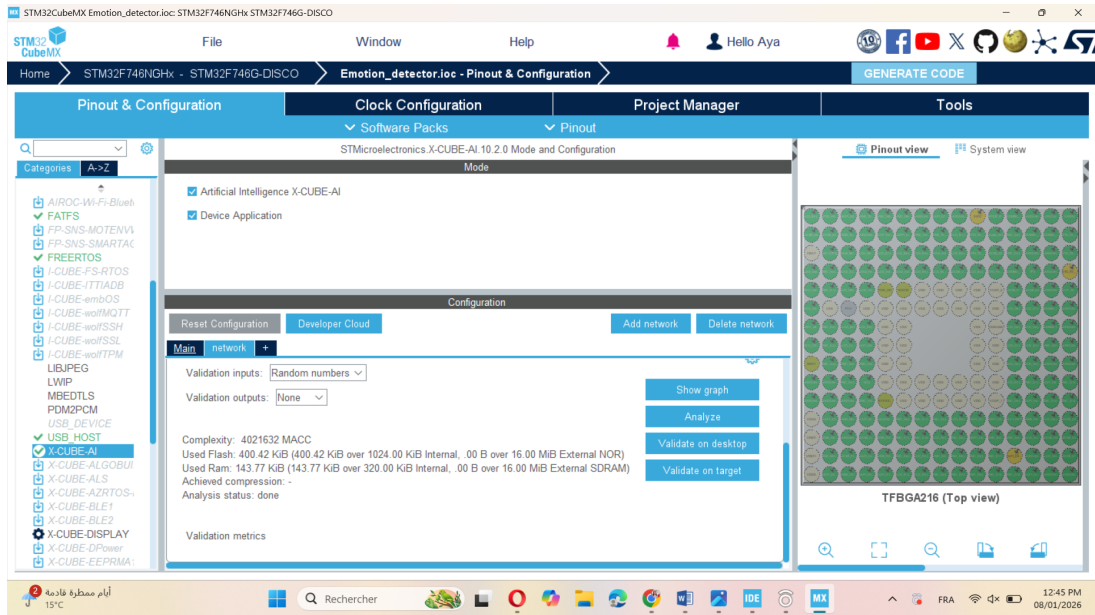


FIGURE 3.4 – Importation et analyse du modèle Keras dans STM32CubeMX

3.4.5 Analyse et validation du modèle

Avant la génération du code, une phase d'analyse est réalisée afin de vérifier la compatibilité du modèle avec la plateforme cible. STM32CubeMX permet également d'effectuer une validation sur la cible (*Validate on desktop*), afin de s'assurer du bon fonctionnement du modèle.

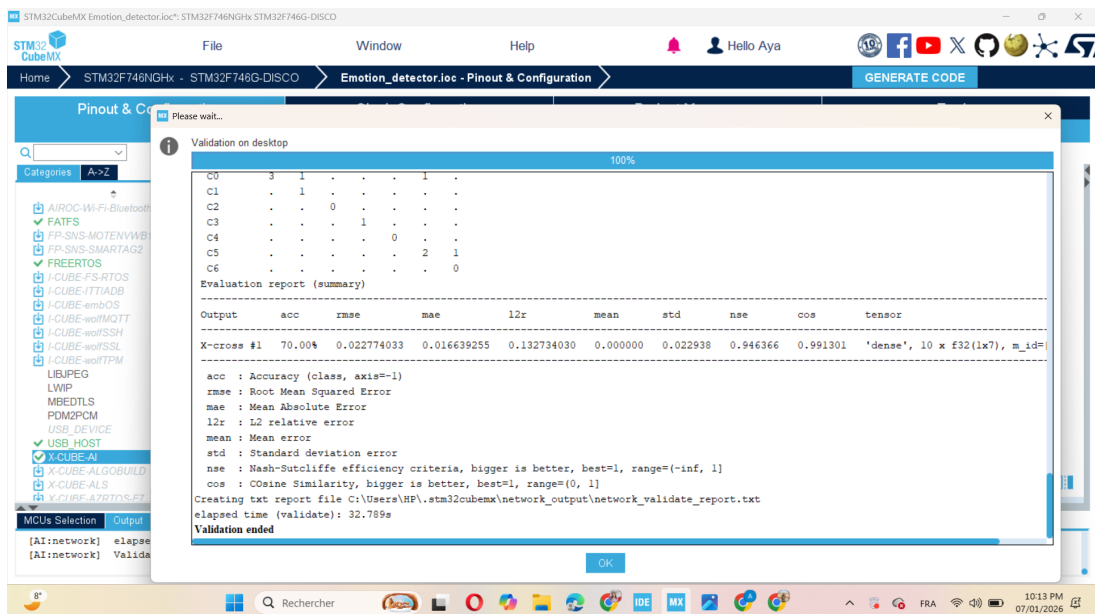


FIGURE 3.5 – Analyse de la complexité et validation du modèle dans STM32CubeMX

3.4.6 Génération du code

Une fois la configuration validée, STM32CubeMX génère automatiquement le code du projet. Ce code inclut l'initialisation du microcontrôleur, les fonctions d'inférence du modèle d'intelligence artificielle ainsi que les structures de données associées au réseau de neurones.

3.5 Exploitation du projet dans STM32CubeIDE

3.5.1 Importation, compilation et flashage

Après la génération automatique du code par STM32CubeMX, le projet est ouvert dans l'environnement de développement **STM32CubeIDE**. Cette étape permet de finaliser l'application embarquée, de compiler le code source et de flasher le programme sur la carte STM32F746G-DISCO.

Lors de l'importation du projet, STM32CubeIDE reconnaît automatiquement la configuration matérielle de la carte ainsi que les fichiers générés par le middleware X-CUBE-AI. Le projet contient notamment :

- les fichiers d'initialisation du microcontrôleur ;
- les fichiers générés par X-CUBE-AI (réseau, poids et buffers) ;
- le code applicatif principal (`main.c`).

La compilation est réalisée sans modification du code généré automatiquement, afin de garantir la compatibilité avec la configuration définie dans STM32CubeMX. Le programme est ensuite flashé sur la carte via l'interface *ST-Link* intégrée.

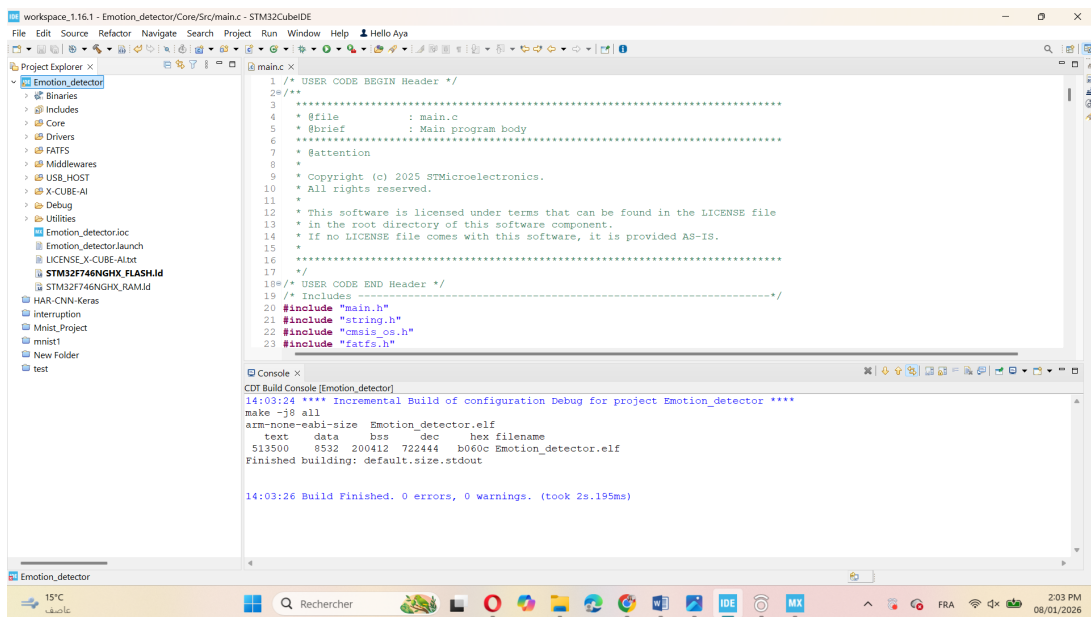


FIGURE 3.6 – Projet généré par STM32CubeMX compilé avec succès dans STM32CubeIDE

3.5.2 Configuration de la communication série (UART1)

Afin de visualiser les résultats de l'inférence du modèle d'intelligence artificielle, une liaison série UART est utilisée. Sur la carte STM32F746G-DISCO, l'interface **USART1** est configurée pour assurer la communication avec un terminal externe.

Les paramètres de communication série sont définis comme suit :

- vitesse de transmission : 115200 bauds ;
- format des données : 8 bits ;
- parité : aucune ;
- bit de stop : 1 ;
- contrôle de flux : désactivé.

Ces paramètres sont configurés dans STM32CubeMX puis utilisés automatiquement dans STM32CubeIDE. Le terminal série **PuTTY** est ensuite employé pour afficher les résultats envoyés par le microcontrôleur.

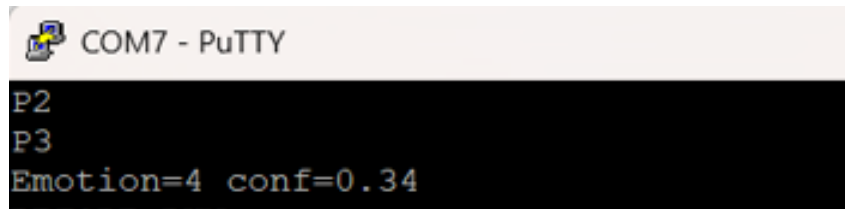


FIGURE 3.7 – Affichage des résultats d'inférence via l'interface UART sur PuTTY

3.5.3 Exécution de l'inférence avec image statique

Lors de l'exécution du programme, l'image de test, préalablement convertie en tableau C, est chargée en mémoire et transmise au modèle embarqué. Cette image constitue l'entrée du réseau de neurones convolutif.

Le processus d'inférence se déroule selon les étapes suivantes :

- copie des données de l'image dans le buffer d'entrée du réseau ;
- exécution de la fonction d'inférence générée par X-CUBE-AI ;
- récupération des probabilités associées à chaque classe d'émotion ;
- identification de la classe présentant la probabilité maximale.

Les probabilités calculées sont ensuite envoyées via l'interface UART et affichées sur le terminal PuTTY, ce qui permet de vérifier le bon fonctionnement du modèle et d'analyser les sorties numériques.

3.6 Amélioration : affichage du résultat sur l'écran LCD

Après validation du fonctionnement du modèle à l'aide de la communication série, une amélioration du système a été réalisée afin d'afficher directement l'émotion prédite sur l'écran LCD embarqué de la carte STM32F746G-DISCO.

3.6.1 Initialisation de l'écran LCD

L'écran LCD est initialisé à l'aide des bibliothèques BSP fournies par STMicroelectronics. Les fonctions de base permettent d'initialiser l'écran, de définir la couleur de fond et d'afficher du texte.

Les principales étapes d'initialisation comprennent :

- l'appel à la fonction d'initialisation du LCD ;
- la configuration de la couche graphique ;
- le nettoyage de l'écran et le positionnement du curseur.

Cette initialisation est effectuée dans la fonction principale du programme, avant l'exécution de l'inférence.

3.6.2 Affichage de l'émotion prédite

Une fois l'inférence terminée, l'émotion correspondant à la probabilité maximale est convertie en une chaîne de caractères. Cette chaîne est ensuite affichée sur l'écran LCD à l'aide des fonctions BSP.

L'affichage du résultat sur l'écran permet :

- une visualisation immédiate de l'émotion prédite ;
- une utilisation autonome du système sans terminal externe ;
- une meilleure ergonomie de l'application embarquée.

3.7 Conclusion

Ce chapitre a présenté de manière détaillée l'ensemble des étapes de déploiement et d'exploitation du modèle de détection d'émotions faciales sur la carte STM32F746G-DISCO. L'utilisation d'images statiques converties en tableaux C a permis de tester le modèle de façon progressive et contrôlée, en commençant par l'analyse des résultats via une interface série, puis en évoluant vers un affichage direct sur l'écran LCD.

Les résultats obtenus démontrent la faisabilité de l'exécution d'un modèle de deep learning sur un microcontrôleur à ressources limitées, à condition d'adapter le modèle et l'architecture logicielle. Dans cette version du système, l'inférence est réalisée à partir d'images statiques afin de valider le fonctionnement du modèle avant une éventuelle intégration d'une caméra.

CONCLUSION GÉNÉRALE

Ce projet avait pour objectif de concevoir et de déployer un système de détection d'émotions faciales basé sur des techniques d'intelligence artificielle, tout en respectant les contraintes spécifiques d'un environnement embarqué. À travers les différentes étapes présentées dans ce rapport, depuis la définition du problème jusqu'au déploiement sur un microcontrôleur STM32, il a été possible de mettre en œuvre l'ensemble du cycle de vie d'un projet de machine learning appliqué.

La phase de conception a permis de sélectionner une architecture de réseau de neurones convolutif adaptée à la reconnaissance d'images, tout en tenant compte des limitations en mémoire et en puissance de calcul. Le travail réalisé sur la préparation des données, l'entraînement du modèle et l'ajustement des hyperparamètres a mis en évidence l'importance d'un compromis entre performance algorithmique et complexité du modèle, en particulier dans un contexte embarqué.

Le déploiement du modèle sur la plateforme STM32, à l'aide des outils STM32CubeIDE et STM32Cube.AI, a constitué une étape déterminante du projet. Il a permis de transformer un modèle entraîné hors ligne en une solution opérationnelle capable de fonctionner en temps réel sur un système à ressources limitées. Les résultats obtenus démontrent que la reconnaissance d'émotions faciales sur microcontrôleur est techniquement réalisable, malgré une légère dégradation des performances par rapport à une exécution sur ordinateur classique.

Enfin, ce travail ouvre des perspectives d'amélioration, telles que l'optimisation supplémentaire du modèle, l'intégration de techniques de quantification avancées ou l'utilisation de capteurs plus performants. Il constitue ainsi une base solide pour le développement futur d'applications embarquées intelligentes.

Bibliographie

- STMicroelectronics, STM32Cube.AI Documentation.
- TensorFlow & Keras Documentation.
- Goodfellow et al., *Deep Learning*, MIT Press.
- Dataset FER2013.